When and Why Does Hindsight Experience Replay Improve Soft Actor-Critic under Sparse Rewards?

Mustafa Asfari
Mohammad Alkhozami
github.com/mustafa99705/-adrl-sac-her-fetchreach

Abstract

We study when and why Hindsight Experience Replay (HER) helps Soft Actor-Critic (SAC) under sparse rewards, comparing four conditions (dense, dense+HER, sparse, sparse+HER) on a simple reaching task (`FetchReach-v3`) and a harder pushing task (`FetchPush-v3`), 5–10 seeds/condition. On the easy task all four conditions solve it near-perfectly; HER only shortens time-to-competence ($2.75\times$ fewer steps to 90% success). On the hard task, plain sparse-reward SAC never learns (5–6% final success, 0 seeds above 90%), while HER on the identical sparse signal reaches 83–91% success — confirming that HER's benefit grows sharply with task difficulty. Tracking SAC's auto-tuned entropy coefficient offers a plausible explanation: without HER it collapses toward zero while the policy is still failing (premature convergence), whereas with HER it stays an order of magnitude higher throughout training. A relabeling-ratio ablation ($n_{\text{sampled\_goal}} \in \{1, 4, 8\}$) shows this benefit saturates rather than growing indefinitely with more relabeling.

1 Introduction

In real robotic tasks, the most natural reward specification is often binary: the task was either accomplished or it was not. Dense reward functions can provide more feedback, but they usually have to be hand-engineered, require domain knowledge, and may bias the learned behavior in unintended ways. However, standard off-policy actor-critic methods such as Soft Actor-Critic (SAC) can struggle under sparse rewards, because the replay buffer may contain many uninformative failed transitions before the agent observes its first success. While Hindsight Experience Replay (HER) has been shown to improve learning in sparse-reward settings, it is less clear under which task conditions its benefit is most significant, and through which mechanism it acts. This report investigates not only *whether* HER improves SAC, but *when* its contribution becomes meaningful as task difficulty increases, and *why* it helps, by analyzing the training dynamics it induces.

2 Related Work

This project connects three main topics from the lecture and the RL literature: off-policy maximum-entropy reinforcement learning, goal-conditioned reinforcement learning, and reward design under sparse feedback. SAC (1) is the off-policy actor-critic algorithm used as the learning backbone, while HER (2) provides the goal-relabeling mechanism that allows failed trajectories to be reused with achieved goals. The project also relates to Universal Value Function Approximators (4) and multi-goal RL, because both the policy and the value function must condition on the desired goal. The Fetch environments and their sparse/dense reward variants are based on the multi-goal robotics benchmark introduced by Plappert et al. (3) and are available through Gymnasium-Robotics.

3 Contribution and Attribution

We build on existing, unmodified implementations rather than reimplementing published algorithms: SAC and its `HerReplayBuffer` are Stable-Baselines3's (6), the Fetch environments and their sparse/dense reward variants are Gymnasium-Robotics' (following Plappert et al. (3)), and our hyperparameters are the tuned Fetch settings from RL Baselines3 Zoo (5) — we did not run our own hyperparameter search. Reimplementing SAC/HER from scratch was explicitly out of scope for this project.

Our own contribution is the experimental design and everything needed to run, diagnose, and interpret it: (1) identifying and fixing a reset bug in `gymnasium-robotics` 1.3.x that silently made the task harder than documented (`env_fix.py`, Section 5); (2) the training/evaluation pipeline (`train.py`) supporting both tasks, all four conditions, and the relabeling-ratio ablation under one shared configuration; (3) `HerDiagnosticsCallback`, which logs the entropy-coefficient and goal-distance-percentile trajectories used for the mechanism analysis in Section 6.2 — this diagnostic is not part of Stable-Baselines3 and was written for this project; (4) designing and running all 70 training runs, plus the separate 2-run pilot used to calibrate the FetchPush step budget, and the analysis/plotting pipeline (`plot_results.py`); and (5) the interpretation in Section 6, including the entropy-collapse account of why HER helps and the finding that relabeling-ratio benefit saturates.

4 Approach

We compare four conditions — dense SAC, dense SAC+HER, sparse SAC, and sparse SAC+HER — on `FetchReach-v3`, a relatively simple reaching task. On `FetchPush-v3`, where the robot must push an object to a target location and sparse-reward learning is known to be substantially harder (2), we focus on the two central sparse-reward conditions, sparse SAC and sparse SAC+HER. Hyperparameters are identical across task, reward type, and HER on/off, so that the reward signal / relabeling strategy is the only experimental variable within each comparison. We tested three hypotheses stated in the project proposal: (i) on the simple reaching task, HER provides only a modest speedup, since random exploration occasionally reaches the goal on its own; (ii) on the harder pushing task, HER is expected to substantially improve learning performance; (iii) HER's effect is visible in the training dynamics, e.g. in the evolution of SAC's entropy coefficient and in the distribution of relabeled goals, forming an implicit curriculum. Section 6 strongly supports hypotheses (i) and (ii) — for (ii), an effect considerably larger than "substantial": HER turned out to be the difference between the task being learned at all — and provides partial evidence for (iii).

Algorithm 1 SAC with optional Hindsight Experience Replay

Require: environment e, SAC algorithm A, reward function r, HER flag h, relabels per transition k
Ensure: trained goal-conditioned policy $\pi(a \mid s, g)$

```
Initialize replay buffer R, SAC actor and critic networks
while training budget not reached do
    Sample goal g and initial state s_0 from e
    Roll out one episode (s_0, a_0, ..., s_T) with π(· | s, g)
    Store (s_t, a_t, r(s_t, a_t, g), s_{t+1}, g) in R for all t
    if h is true then
        for each t and each of k relabels do
            sample τ ~ U(t+1, T);  g' ← achieved goal in s_τ
            store (s_t, a_t, r(s_t, a_t, g'), s_{t+1}, g') in R      ▷ often a success: r = 0
        end for
    end if
    Update SAC actor and critics on minibatches from R
end while
return π
```

Algorithm 1 illustrates HER conceptually. In our Stable-Baselines3 implementation, only real transitions are stored in R; relabeled (virtual) transitions are generated on the fly when minibatches are sampled from `HerReplayBuffer`, not written to R at rollout time (Section 6.2).

5 Experimental Setup

Both tasks use Stable-Baselines3 SAC (6) with a `MultiInputPolicy` for the dictionary observations, and share one fixed hyperparameter configuration adopted from the tuned RL Baselines3 Zoo settings for Fetch-style tasks (5) ($\gamma = 0.95$, learning rate 10^{-3}, batch size 256, replay buffer size 10^6, HER $n_{\text{sampled\_goal}} = 4$ with the *future* strategy where applicable; all remaining settings are Stable-Baselines3 defaults (6)). We deliberately did not tune hyperparameters per condition, to keep the reward signal / HER as the only variable. The success criterion is the environment's own (end effector, respectively object, within 5 cm of the goal at episode end).

We found and fixed a reset bug in `gymnasium-robotics` 1.3.x (the only release line exposing the `-v3` Fetch environments): the initial arm pose was not restored on reset, so every episode started with the gripper $\sim$0.55 m from the goal-sampling center instead of the documented $\leq$0.26 m — a $\sim$2.1$\times$ larger initial offset ($0.55/0.26$). The fix (`env_fix.py`) backports the official upstream correction and is applied identically to every run below.

Scope actually run. The proposal specified 5 seeds per condition and an ablation over $n_{\text{sampled\_goal}} \in \{1, 4\}$. We ran a larger design instead: 10 seeds per condition throughout, and a third ablation setting $n_{\text{sampled\_goal}} = 8$, to get tighter confidence bands on FetchPush's noisier seeds (Section 6 shows this variance was substantial and worth the extra runs). Concretely: on `FetchReach-v3` we ran all four conditions with 10 seeds each (100,000 steps/run, evaluating 20 held-out episodes every 2,000 steps). On `FetchPush-v3` we ran the two central conditions, sparse and sparse+HER, with 10 seeds each. A short pilot (1 seed/condition, 200,000 steps) showed FetchPush success was still near zero at that budget (0% sparse, 5% sparse+HER) and measured a throughput of 54–61 steps/s; we used this to set the main-grid budget to 10^6 steps/run (matching the proposal's estimate) and evaluated every 10,000 steps rather than the 2,000 used for Reach, to keep evaluation overhead proportionate to the $10\times$ longer budget. For the ablation, we ran $n_{\text{sampled\_goal}} \in \{1, 4, 8\}$ on `FetchPush-v3` with 5 seeds per setting ($n_{\text{sampled\_goal}} = 4$ reuses the first 5 seeds of the main sparse+HER condition rather than re-running it). In total the study comprises 70 training runs, all completed successfully. To probe *why* HER helps, every run additionally logged SAC's auto-tuned entropy coefficient α and the distribution of $\|\text{achieved goal} - \text{desired goal}\|$ in sampled replay-buffer minibatches, both every 2,000 (Reach) or 10,000 (Push) steps.

6 Results

6.1 Main comparison: does HER's benefit grow with task difficulty?

Metric definitions used in all tables below: per-seed *success* is the mean evaluation success rate over the last 5 evaluation points of a run; *Mean/Median success* aggregate this across seeds within a condition (we report both because Push has occasional catastrophic-failure seeds that pull the mean down without affecting the median – see Section 7). *Median steps to 90%* is computed only over seeds whose evaluation curve reaches 90% success at some point; the count of seeds that do so is reported separately in the *Seeds $\geq$90%* column, and seeds that never reach it are excluded from the median (not treated as ∞). *AUC* is the evaluation success-rate curve integrated (trapezoidal rule) over training steps and normalized by the total training budget, so $\text{AUC} \in [0, 1]$.

Table 1: As specified in the proposal: 5 seeds/condition.

Task	Condition	Mean success	Median success	Seeds $\geq$90%	Steps to 90%	AUC
Reach	dense	100.0%	100.0%	5/5	6,000	0.944
Reach	dense + HER	100.0%	100.0%	5/5	6,000	0.945
Reach	sparse	98.8%	100.0%	5/5	22,000	0.804
Reach	sparse + HER	100.0%	100.0%	5/5	8,000	0.931
Push	sparse	6.2%	6.0%	0/5	— (never)	0.065
Push	sparse + HER	82.8%	99.0%	4/5	380,000	0.553

At the protocol's originally-specified scale, the central finding is already unambiguous: HER is a mild accelerant on Reach and the difference between learning and not learning on Push. Tables 2 and 3

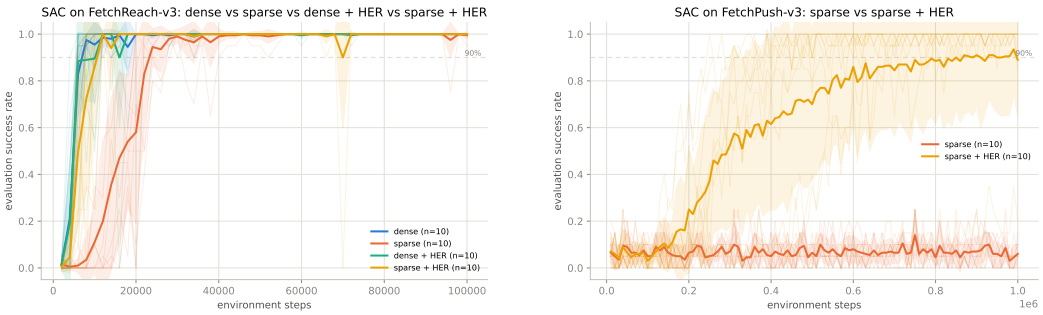
below extend every condition to 10 seeds — purely to tighten the confidence bands on Push's noisier seeds, discussed in Section 6.2 — and show the conclusion is unchanged, not seed-count-dependent.

Table 2: Extended robustness check: same runs, 10 seeds/condition (FetchReach-v3). All conditions solve the task; HER mainly affects how fast.

Condition	Mean success	Median success	Seeds $\geq$90%	Steps to 90%	AUC
dense	100.0%	100.0%	10/10	6,000	0.947
dense + HER	100.0%	100.0%	10/10	6,000	0.945
sparse	99.4%	100.0%	10/10	22,000	0.826
sparse + HER	100.0%	100.0%	10/10	8,000	0.929

Table 3: Extended robustness check: same runs, 10 seeds/condition (FetchPush-v3). Sparse SAC essentially fails to learn; HER recovers most of the task.

Condition	Mean success	Median success	Seeds $\geq$90%	Steps to 90%	AUC
sparse	5.4%	6.0%	0/10	— (never)	0.067
sparse + HER	90.7%	99.0%	9/10	350,000	0.610



(a) FetchReach-v3

(b) FetchPush-v3

Figure 1: Evaluation success rate vs. environment steps (thin lines: individual seeds; thick line: mean; band: mean $\pm$ 1 std. dev. across seeds). On Reach, all HER and dense conditions overlap and converge fast, with plain sparse trailing but still reaching near-perfect success. On Push, sparse SAC (orange) never rises above noise around 5–10%, while sparse+HER (yellow) climbs steadily to $\sim$90%.

Table 2 confirms hypothesis (i): on the easy task, every condition eventually succeeds, and HER's contribution is a $\sim$2.75$\times$ reduction in steps-to-90% for the sparse reward (22,000 $\rightarrow$ 8,000), not the difference between success and failure. Table 3 and Figure 1b confirm hypothesis (ii) sharply: without HER, SAC extracts essentially nothing from the sparse pushing reward over a full million steps (5.4% final success, close to the pre-training baseline), while HER on the identical reward signal reaches 90.7% mean success (99.0% median — the gap between the two is one catastrophic-failure seed, discussed in Section 7). HER's benefit is not a speedup here — it is the difference between the task being learned at all.

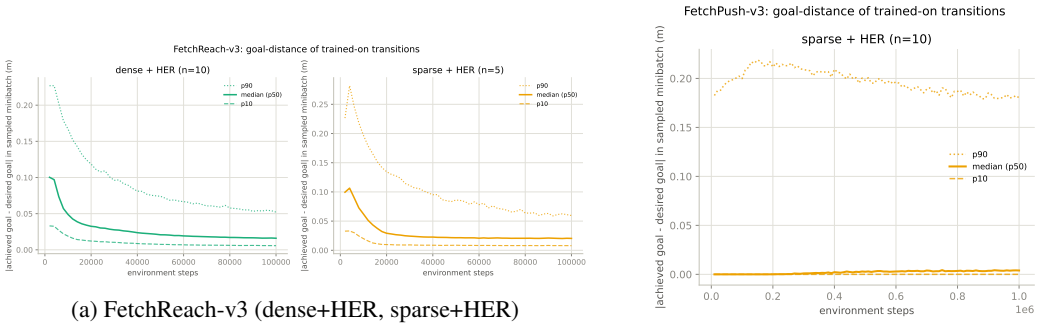
6.2 Mechanism: why does HER help?

Figure 2 is consistent with a plausible mechanistic account of hypothesis (ii): SAC's entropy term is auto-tuned to keep the policy just exploratory enough to keep improving. Under plain sparse reward on Push, the critic receives very few rewarding transitions to correct its estimates (only 5.4% of episodes ever succeed), so SAC settles into an overconfident, low-entropy policy despite *not* having solved the task — a premature-convergence failure mode. With HER supplying relabeled successful transitions from the start, the critic receives a much denser learning signal, and α is kept an order of magnitude higher for the full training run, coinciding with the exploration needed to eventually discover successful pushes. Notably, entropy collapse *per se* is not the problem: on Reach (Figure 2a),

(a) FetchReach-v3 (b) FetchPush-v3

Figure 2: SAC's auto-tuned entropy coefficient α (log scale, mean $\pm$ 1 std. dev. over 10 seeds). On Reach (a), α collapses for *every* condition ($\approx 0.368 \rightarrow \approx 3.1$–$3.2 \times 10^{-4}$) in step with the task being solved — a healthy collapse. On Push (b), without HER α collapses $\sim$21$\times$ ($6.2\times10^{-3} \rightarrow 2.9\times10^{-4}$) *while the policy is still failing*; with HER it stays roughly flat around 2.2–2.3 $\times 10^{-3}$, about 8$\times$ higher than sparse's final value — the same collapse pattern, but only healthy in one of the two cases.

α collapses similarly for every condition (from ≈ 0.368 to ≈ 3.1–3.2×10^{-4}), but there it coincides with the task actually being solved — collapse tracks genuine competence in one case and precedes it in the other. We note this evidence is an association between HER and healthier entropy dynamics, not a controlled test that isolates entropy as the causal mechanism.



(a) FetchReach-v3 (dense+HER, sparse+HER)

(b) FetchPush-v3 (sparse+HER)

Figure 3: p10/median/p90 of $\|\text{achieved} - \text{desired goal}\|$ across transitions actually sampled for training, plotted as three separate lines (rather than a shaded band) so a median pinned near zero does not visually mask movement in the upper percentiles.

For the implicit-curriculum part of hypothesis (iii), the picture differs by task. On Reach (Figure 3a), the median trained-on goal distance falls cleanly as training progresses — from $\approx$0.10 m to $\approx$0.016–0.020 m for dense+HER and sparse+HER respectively — a direct, visible curriculum: as the policy improves, the goals it is actually trained on get closer on average. On Push (Figure 3b) the same signal is present but far less visible in this aggregate view: with $n_{\text{sampled\_goal}} = 4$, roughly 80% of *sampled* training transitions are HER-relabeled (empirically distance $\approx$0 for most of these, per `HerDiagnosticsCallback`'s measured percentiles), which pins the median near zero for the entire run and hides the curriculum in the remaining $\sim$20% of real-goal transitions. Those real-goal transitions do show a mild version of the same effect: since p10 and the median both stay pinned near zero throughout (dominated by relabeled samples), it is the *p90 line* specifically that carries this signal — rising from $\approx$0.18 m to a peak of $\approx$0.21–0.215 m by $\sim$150–250k steps as the policy starts reaching more diverse states, then falling back to $\approx$0.18 m by 1M steps — but it is a second-order effect on Push, not the dominant pattern visible in the raw metric. We report this as an honest limitation of the metric rather than evidence against hypothesis (iii): the entropy-coefficient signal (Figure 2) is the clearer mechanism indicator on the harder task.

6.3 Ablation: does more relabeling always help?

The proposal specifies this ablation over $n_{\text{sampled\_goal}} \in \{1, 4\}$, 5 seeds/setting; we additionally ran $n_{\text{sampled\_goal}} = 8$ at the same 5 seeds/setting to check whether the trend continues past the specified range.

Table 4: HER relabeling-ratio ablation, FetchPush-v3 sparse+HER, 5 seeds/setting (rows 1–4 are the specified range; row 8 is the extension).

$n_{\text{sampled\_goal}}$	Mean success	Median success	Seeds $\geq$90%	Steps to 90%	AUC
1	84.0%	99.0%	4/5	400,000	0.566
4	82.8%	99.0%	4/5	380,000	0.553
8	88.6%	99.0%	4/5	260,000	0.637

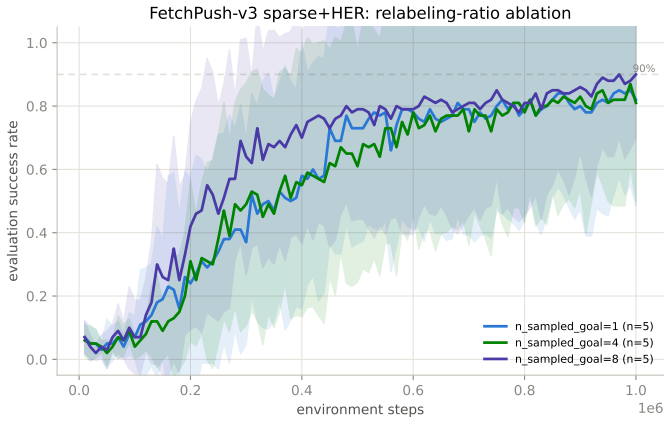


Figure 4: Success rate by HER relabeling ratio on FetchPush-v3 (mean over 5 seeds, band: mean $\pm$ 1 std. dev.).

Table 4 and Figure 4 address the required ablation: relabeling ratio does not help monotonically or dramatically. $n_{\text{sampled\_goal}} = 8$ learns somewhat faster (median steps-to-90% 260,000 vs. $\sim$380–400,000) and reaches a marginally higher final success and AUC, but $n_{\text{sampled\_goal}} = 1$ and 4 show no clear separation with the available five seeds, and all three settings converge to a similar success region by 1M steps. Every setting also has exactly one seed out of five that fails to reach 90% (final success 0.20–0.45 for that seed, vs. $\geq$0.96 for the other four in the same group; visible as the wide lower band in Figure 4) — suggesting per-seed optimization instability on this harder task is a bigger source of variance than the relabeling ratio itself. This is underscored by Table 4's median column: the median final success is identically 99.0% for $n_{\text{sampled\_goal}} \in \{1, 4, 8\}$ — the mean differences between settings are driven entirely by which single seed happened to fail, not by a real difference in typical performance. We conclude the benefit of more relabeling saturates quickly on this task rather than scaling with k.

7 Discussion and Limitations

The central result — HER's benefit growing from a modest speedup on FetchReach to the difference between learning and not learning on FetchPush — strongly supports hypotheses (i) and (ii) from the proposal and provides partial support for (iii), and the entropy-coefficient trajectories (Section 6.2) offer a plausible mechanistic account of *why*: HER is associated with avoiding the premature, overconfident entropy collapse that plain sparse-reward SAC falls into when it rarely observes a success. We note three limitations. First, both FetchPush conditions show occasional catastrophic-failure seeds (final success as low as 0.20 despite HER); with only 10 (main grid) or 5 (ablation) seeds per setting, these outliers noticeably pull down the mean without affecting the median (Tables 3 and 4 report both for exactly this reason). Second, the goal-distance curriculum signal, clear on Reach, is largely masked on Push by the relabeling ratio itself (Section 6.2); a cleaner mechanism

metric for harder tasks would track only the non-relabeled ("real-goal") transitions directly rather than the pooled distribution. Third, we deviated from the proposal's literal evaluation cadence (every 2,000 steps) on Push, using 10,000 steps instead to keep evaluation overhead proportionate to the $10\times$ longer training budget; this does not affect the reported metrics' correctness but is a departure from the originally specified protocol worth flagging explicitly.

8 Reproducibility

All code, per-run logs/metrics/configs, and this report's source are in the project repository (see title page link); trained model checkpoints (excluded from the repository as large binaries, $\sim$540MB across 70 runs) are available on request. `README.md` documents the software stack (pinned for the `gymnasium-robotics` 1.3.x / `-v3` environments) and exact commands to reproduce every run; `EXPERIMENT_LOG.md` records the full chronological history of decisions, calibration numbers, and cluster job IDs behind the numbers in this report. `plot_results.py` regenerates all figures and summary tables directly from the tracked per-run data. All 70 training runs (plus a 2-run pilot) were executed on the GWDG/KISSKI HPC cluster, one NVIDIA A100 GPU per run, 4 CPU cores, 16GB RAM; total compute used was $\approx$115 GPU-hours.

9 Conclusion

This report set out to answer not just *whether* HER helps SAC under sparse rewards, but *when* and *why* — both questions are answered directly by the 70 runs above. *When*: HER's contribution to SAC under sparse rewards is not fixed, it scales with task difficulty — a modest accelerant on an easy task, the difference between success and failure on a harder one. *Why*: this shift is accompanied by markedly different entropy dynamics — without HER, SAC's entropy coefficient collapses while the policy is still failing, whereas with HER it stays substantially higher throughout training, coinciding with successful learning rather than premature convergence on a failing behavior. This evidence is consistent with, but does not by itself establish, a causal exploration mechanism. More hindsight relabeling helps up to a point but saturates quickly, making $n_{\text{sampled\_goal}} = 4$–$8$ a reasonable default rather than a setting worth aggressively tuning further.

Natural next steps: (i) re-run the mechanism analysis tracking only non-relabeled transitions to get a cleaner curriculum signal on Push (Section 6.2); (ii) test whether the difficulty-scaling pattern found here (Reach vs. Push) extends to a third, even harder Fetch task (e.g. `FetchSlide-v3` or `FetchPickAndPlace-v3`); and (iii) a formal significance test (e.g. bootstrap confidence intervals) on the seed-level results, given the catastrophic-failure-seed variance noted in Section 7.

References

[1] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ICML*, 2018.

[2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, W. Zaremba. Hindsight Experience Replay. *NeurIPS*, 2017.

[3] M. Plappert, M. Andrychowicz, A. Ray, B. McGrew, B. Baker, G. Powell, J. Schneider, J. Tobin, M. Chociej, P. Welinder, V. Kumar, W. Zaremba. Multi-Goal Reinforcement Learning: Challenging Robotics Environments and Request for Research. *arXiv:1802.09464*, 2018.

[4] T. Schaul, D. Horgan, K. Gregor, D. Silver. Universal Value Function Approximators. *ICML*, 2015.

[5] A. Raffin. RL Baselines3 Zoo: A Training Framework for Stable-Baselines3 Reinforcement Learning Agents. `https://github.com/DLR-RM/rl-baselines3-zoo`, 2020.

[6] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*, 22(268), 2021.

Checklist

This checklist is not mandatory, but can help you set up your project in a reproducible manner.

1. General points:
   (a) Do the main claims made in the abstract and introduction accurately reflect your contributions and scope? [Yes]
   (b) Did you cite all relevant related work? [Yes]
   (c) Did you describe the limitations of your work? [Yes] Section 7
   (d) Did you include a discussion of future work? [Yes] Section 6 end / Conclusion
2. If you are including theoretical results...
   (a) Did you state the full set of assumptions of all theoretical results? [NA] [purely empirical study, no theoretical claims]
   (b) Did you include complete proofs of all theoretical results? [NA]
3. If you ran experiments (e.g. for benchmarks)...
   (a) Did you include the code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL)? [Yes] [GitHub repository linked on the title page; README.md and EXPERIMENT_LOG.md document exact commands and decisions]
   (b) Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)? [Yes] [Section 5; hyperparameters taken from RL Baselines3 Zoo, no search performed – Section 3]
   (c) Did you run at least 10 repetitions of your method? [Yes] [proposal specified 5 seeds/condition; we ran 10/condition for both main comparisons and 5/setting for the ablation (as specified)]
   (d) Did you report error bars (e.g., with respect to the random seed after running experiments multiple times)? [Yes] [figures show per-seed traces and mean $\pm$ range/p10–p90 bands; per-seed outlier variance discussed explicitly in Section 7]
   (e) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [Yes] [Section 8: $\approx$115 GPU-hours, 1$\times$ NVIDIA A100/run, GWDG/KISSKI HPC cluster]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets...
   (a) If your work uses existing assets, did you cite the creators? [Yes] [Section 3: Stable-Baselines3, Gymnasium-Robotics, RL Baselines3 Zoo]
   (b) Did you make sure the license of the assets permits usage? [Yes] [Stable-Baselines3 and Gymnasium-Robotics are both MIT-licensed; standard academic/educational use]
   (c) Did you reference the assets directly within your code and repository? [Yes] [exact versions pinned in `requirements.txt`; imports visible in `train.py`]